Lab 2: Graphs and Topological Sort

Lab 2 examines the use of the Graph data structure discussed in Lectures and lays the groundwork for the major assignment.

You are supplied with the `Graph` abstract class that defines a number of essential methods for operating on graphs.

Checkpoints

Checkpoint L2.1: Adjacency List Graph Implementation

You are required to create an implementation for the `Graph` interface that uses an **adjacency list** as the underlying data structure. You need to implement both an undirected graph model and a directed graph model, for example

```
Graph g = new AdjacencyListUndirectedGraph();
```

or

```
Graph g = new AdjacencyListDirectedGraph();
```

The main difference between the two implementations is the `addEdge` method. The **undirected** graph will be connected in **both** directions, whereas the **directed** will be connected in only **one** direction, i.e. `addEdge("a","b")` will connect only from vertex a to b.

Your graph data structure needs to maintain a store of the vertices and the edges between them. There are several ways to achieve this and Java gives you many options for data structures (Lists, Maps, Sets, arrays). Think carefully about your choices and how your represent your adjacency “list”. You may be tempted to use a `Map` because of the O(1) lookup, but there are some limitations to the Java Map implementations that **may be** difficult to overcome. It is **strongly** suggested that

you take the conceptually easier choice first and once you have it working you might like to optimise.

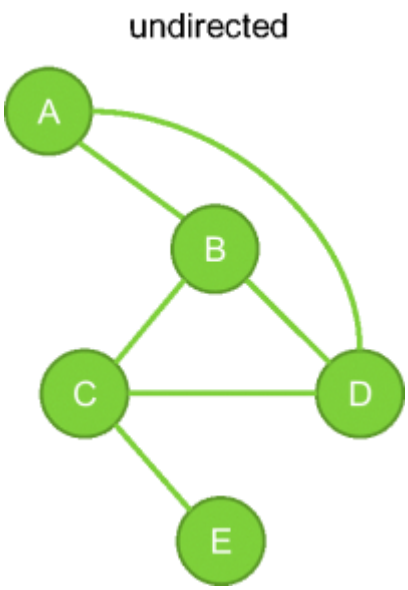
Your methods (e.g. `getVertices()`) for your `Graph` implementations should return `Vertex` objects (or a representation of the `Vertex`) in a sorted way. For this lab, that is **lexicographically** as defined by the `String` class on the label of the `Vertex` (see the `compareTo` method in the `Vertex` class described below).

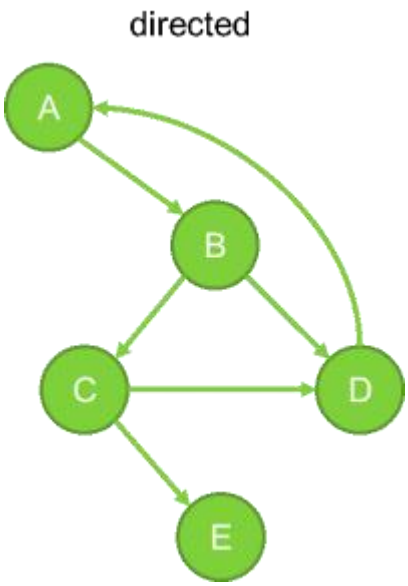
You have been provided with an implementation of the `Vertex` class to represent a vertex in the graph. The `Vertex` class needs to be able to represent the vertex and its data. At this point our vertex can have a label (of type `String`). Depending on your implementation, you might find the implementations of the methods `equals`, `hashcode`, the `Comparable` interface and the supplied `compareTo` method useful. You are welcome to modify this class suit your needs (e.g. add methods and variables for degree, in degree, out degree, etc.)

The `Graph` data structure also needs to maintain knowledge of state of the `Vertex` objects. It needs to maintain the state of being undiscovered, discovered and finished. To enable this a `Graph` class maintains an instance variable of type `Map` called `stateMap` of the states of each `Vertex` it contains. Feel free to choose a different way to represent the states of the `Vertex` objects, but this approach may side step the warning below.

The `GraphDriver` class provides some methods to test out your implementation. The method `test1(Graph g)` builds a graph with the following representation (based on example in

lectures):





For an **undirected** graph, `test1` from the `GraphDriver` class will give an output of (user input denote with `<-- user input`):

```
1 U    <-- user input
test 1: Build a graph
Vertices: [a, b, c, d, e]
a: b d  b: a c d  c: b d
e
d: a b c
```

```
e: c
```

For a **directed** graph, `test1` from the `GraphDriver` class will give an output of:

```
1 D   <-- user input
test 1: Build a graph
Vertices: [a, b, c, d,
e] a:                                                          1

b: c                                                           4

c: dd: a                          Error! Bookmark not defined.


e:
```

You have been provided with a breadth first search class, `BreadthFirstSearch`, that takes a `Graph` and a starting `Vertex` and generates the breadth first traversal list, `getBreadthFirstTraversalList`(), and also maintains search paths from the starting `Vertex` to a given `Vertex`, so that you can get the distance to a particular `Vertex`, `getDistanceTo`, and find the path to that `Vertex` as well, `pathTo`.

The method `test2()` applies the breadth first search to the same graph as above. Your output might differ if the order of your adjacent `Vertices` differs from mine (but it really should not differ!)

For an **undirected** graph, `test2` from the `GraphDriver` class will give an output of:

```
2 U  <-- user input test 2:
Breadth First Search
Vertices: [a, b, c, d, e] a:
b d  b: a c d  c: b d e
d: a b c
```

```
e: c
[a, b, d, c, e] dist
from a to a: 0 dist
from a to b: 1 dist
from a to c: 2
dist from a to d: 1
dist from a to e: 3
```

For a **directed graph**, `test2` from the `GraphDriver` class will give an output of:

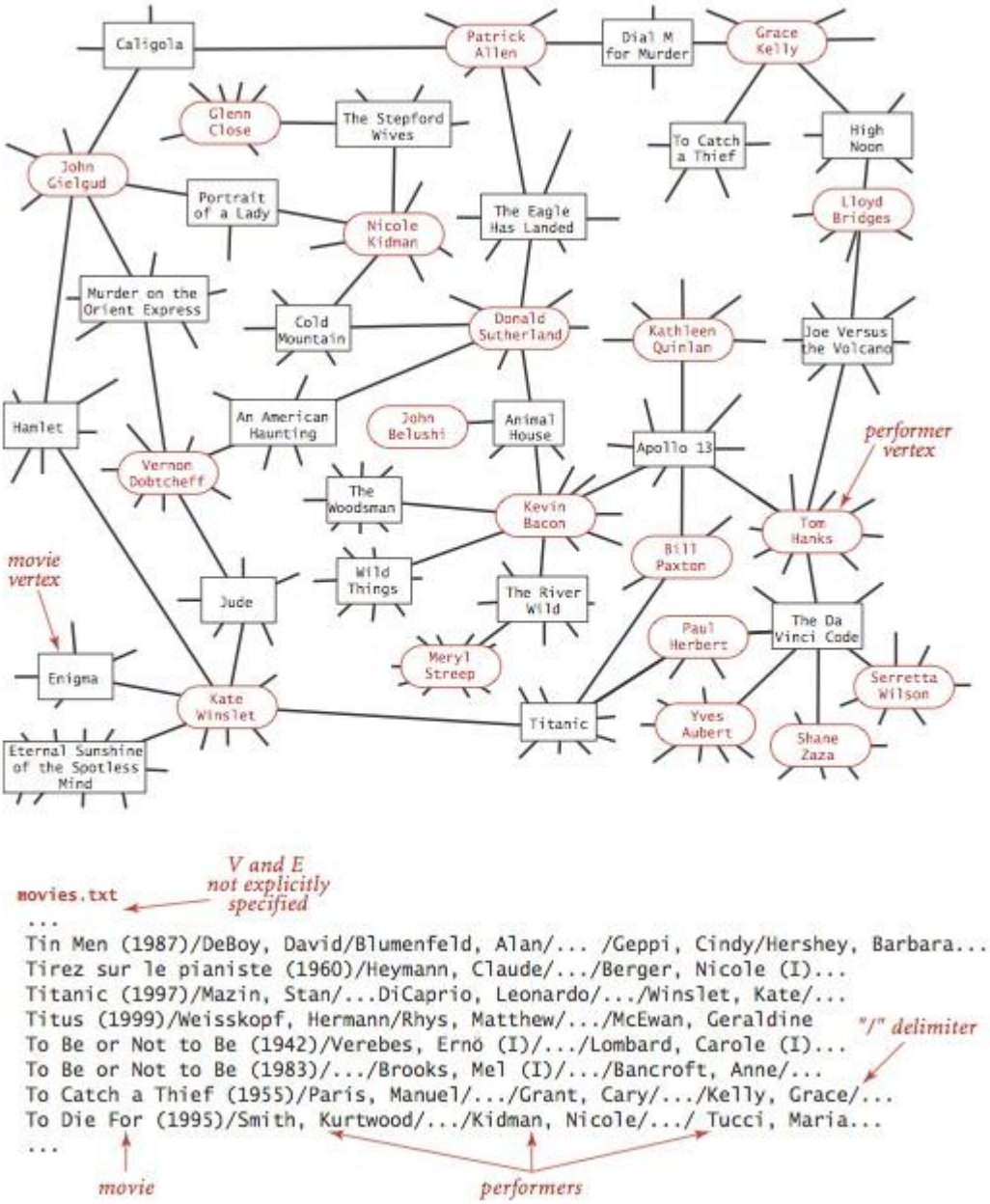
```
2 D    <-- user input test
2: Breadth First Search
Vertices: [a, b, c, d, e] a:
b  b: c d
c: d e
d: a  e:
[a, b, c, d, e] dist
from a to a: 0 dist
from a to b: 1 dist
from a to c: 2 dist
from a to d: 2 dist
from a to e: 3
```

An issue that may arise is determining if a vertex already exists in you `Graph` data structure and ensuring that you only have one instance of particular label, e.g. all the "d"'s refer to the `Vertex` with the label "d".
For example, `GraphDriver.test1` adds the edge ("b","d"), then the edge ("c","d"), and then the edge ("d","a"). You need to ensure that the vertex "d" in those edges are referring to the same object in memory. That is, the "d" in the "b" adjacency list needs to refer to the same "d" in the "c" adjacency list and the "d" in the list of vertices.

Otherwise, when you traverse the graph with a breadth or depth first search and mark the vertices as discovered that

change might not actually be reflected in the "d" object. But it depends on your implementation...you've been warned.

If all that works, you should now be able to run the method `bacon()` from the `GraphDriver` class. This method reads in a file that contains a list of movies and the actors associated with them. This forms an undirected graph with vertices for movies and actors. A movie vertex would have an adjacency list of actors and an actor vertex will have an adjacency list of movies, for example,



Executing the `bacon()` method will result in it reading the `cast.rated.txt` file, building an undirected graph, performing a breadth first search from the actor "`Bacon, Kevin`" and then prompting for another actor, for example (user input indicated with `<-- user input`):

```
5   <-- user input reading in
file: cast.rated.txt calculating
bfs for Bacon, Kevin
```

```
Enter an actor <lastname, firstname>: Winslet, Kate <-- user
input
dist from Bacon, Kevin to Winslet, Kate: 2
  Winslet, Kate
  Titanic (1997)
  Paxton, Bill
  Apollo 13 (1995)
  Bacon, Kevin
Enter an actor <lastname, firstname>:
```

For checkpoint L3.1, show the working examples of the undirected and directed graph adjacency list implementations (no need for the `bacon()`, it is just for fun!)

Checkpoint L2.2: Depth First Search

Implement a **post-order, depth first search/traversal** of your graph data structures keeping track of the vertices traversed. Use the `BreadthFirstSearch` class as a guide as to how to use the `Graph` interface and the `Vertex` and its `Vertex.VertexState` enumeration. Your implementation should iterate through the nodes in **lexicographical** order. That is, in the example below you should start a Vertex "a" and then explore its outgoing edge "b" next.
The method `test3()` applies the depth first search to the same graph as above. Your output might differ if the order of your adjacent Vertices differs from mine (but really it should not differ, remember lexicographical!).

For an **undirected** graph, `test3("U")` from the GraphTester class will give an output of:

```
3 U   <-- user input test
3: Depth First Search
Vertices: [a, b, c, d, e]
a: b d  b: a c d  c: b d e
```

```
d: a b c  e: c
Depth First [d, e, c, b,
a]
```

For a **directed** graph, `test3("D")` from the GraphTester class will give an output of:

```
3 D    <-- user input test
3: Depth First Search
Vertices: [a, b, c, d, e]
a: b  b: c d  c: d e
d: a  e:  Depth First
[d, e, c, b, a]
```

For checkpoint L3.2, show the working examples of the undirected and directed graph adjacency list implementations for the depth first search.

Checkpoint L2.3: Topological Sort

Perform a **topological sort** of a directed graph.

Convert Bob's Language Topic problem from lectures into your graph representation and find a sequence of topics that satisfies the prerequisites. Your implementation should iterate through the nodes in lexicographical order. That is, in the example below you should start a Vertex "LA15" and then explore its outgoing edge "LA16" next.

A potential challenge with this task is that you need to automatically find which vertices to start the search from. In the case of Bob's language topics these are the topic LA15 and LA22. But how do we know that? These two vertices have an **in-degree of 0**, that is there are no edges where these vertices are the "to" vertex. Another way, LA15 does not appear in any other vertices adjacency list.

So, two steps are required for the Topological Sort

1. Find all vertices that have an in-degree of 0
2. For all vertices that have an in-degree of 0, perform a depth first search, maintaining the list of traversed vertices (remembering to traverse in lexicographical order)

A potential output of processing Bob's language topics might look like (note: the sorting has LA126 as "smaller" than LA15, which it is lexicographically):

```
4 Y    <-- user input test
4: Topological Sort
Vertices: [LA126, LA127, LA141, LA15, LA16, LA169, LA22,
LA31, LA32]
LA126:
LA127:
LA141:
LA15: LA16 LA31  LA16:
LA127 LA141 LA32
LA169:
LA22: LA126 LA141
LA31: LA32
LA32: LA126 LA169
Topological Sort
[LA22, LA15, LA31, LA16, LA32, LA169, LA126, LA141, LA127]
```

or just the list

```
4 N   <-- user input test
4: Topological Sort
Topological Sort
[LA22, LA15, LA31, LA16, LA32, LA169, LA126, LA141, LA127]
```

For checkpoint L3.3, show the result of the topological sort for Bob's language topics, helping Bob choose the correct sequence of topics.

Sample GraphDriver Usage

The GraphDriver class expects no command line arguments but processes one line from standard input.

The format is: `level_test additional_args` The `level_test` is a number between 1 and 5.

1. Example usage is print graph of undirected ("U") or directed ("D") graph

```
1 U
1 D
```

2. print graph, the traversal order and distance from a starting node of breadth first traversal of undirected ("U") or directed ("D") graph

```
2 U
2 D
```

3. print graph, the traversal order and distance from a starting node of depth first traversal of undirected ("U") or directed ("D") graph

```
3 U
3 D
```

4. Process the topological sort for Bob's language topics. "Y" to print the graph and "N" to only print the topic list.

```
4 Y
4 N
```

5. Do the Bacon!

```
5
```

For 2 and 3 you can also specify a start vertex to traverse from. The default value is "a", for example,

```
2 U a
```

will be an undirected graph doing a breadth first traversal start from the node labelled with an "a".

```
3 D c
```

will be an directed graph doing a depth first traversal start from the node labelled with an "c".